

Improved Algorithms and Data Structures for Solving Graph Problems in External Memory *

Vijay Kumar [†]
Eric J. Schwabe [‡]

Department of EECS
Northwestern University
Evanston, IL 60208

Abstract

Recently, the study of I/O-efficient algorithms has moved beyond fundamental problems of sorting and permuting and into wider areas such as computational geometry and graph algorithms. With this expansion has come a need for new algorithmic techniques and data structures. In this paper, we present I/O-efficient analogues of well-known data structures that we show to be useful for obtaining simpler and improved algorithms for several graph problems. Our results include improved algorithms for minimum spanning trees, breadth-first search, and single-source shortest paths. The descriptions of these algorithms are greatly simplified by their use of well-defined I/O-efficient data structures with good amortized performance bounds. We expect that I/O-efficient data structures such as these will be a useful tool for the design of I/O-efficient algorithms.

1. Introduction

1.1. Background and model

The study of I/O-efficient algorithms has been receiving increased attention as increases in processor speeds continue to exceed increases in throughput rates between main and external memories. I/O bottlenecks are already a significant concern, and will become an even bigger problem in the future. This problem is all the more pressing in parallel computing and multiprocessing, where narrow I/O channels will be further overtaxed by the data demands of large numbers of active processors.

One of the many domains in which problems that are too large to be solved in main memory are regularly encountered is that of graph algorithms. No matter how large the capacity of the main memory is, there will always be a problem that is too big to fit. When these large problems are solved using parallel processing techniques, as is often the case, I/O-efficiency is even more crucial. Some examples of areas in which such problems arise are the modeling of large phone or communication networks, and large circuit layout problems.

In this paper, we consider algorithms and data structures for solving problems on large graphs that reside in external memory. Our computational model consists of a single processor with a small local memory connected to a large external memory, which may consist of multiple parallel disks. The parameters of the model are as follows:

- N = the total size of the problem in external memory,
- M = the number of items that can fit into the local memory,
- B = the number of items in one block of data, and
- D = the number of parallel disks in the memory system.

It is assumed that $M < N$ and $1 \ll DB < M/2$.

The basic I/O operation (first described by Vitter and Shriver [7]) consists of the reading (or writing) of one block of data from (or to) each of the D disks in parallel — a transfer of a total of DB items between main memory and external memory. The address space of the external memory is *striped* block-wise across the D disks — that is, the r th block overall of the N/B blocks of data is the $(\lfloor r/D \rfloor)$ th block on disk number $r \bmod D$. Our goal will be to solve problems using as few of these I/O operations as possible.

*This research was supported in part by the National Science Foundation under grant CCR-9309111.

[†]Author's email address: vijay@eecs.nwu.edu

[‡]Author's email address: schwabe@eecs.nwu.edu.

After Chiang et al. [3], we define the useful quantities $scan(x) = \frac{x}{DB}$ and $sort(x) = \frac{x}{DB} \log_{M/B} \frac{x}{B}$. Up to constant factors, $scan(x)$ is the number of I/O's required to read x consecutive items stored on the D disks (recall that the ordering is striped across the disks), and $sort(x)$ is number of I/O's that is necessary and sufficient to sort x items stored consecutively on the D disks [5, 6, 7].

Since we will be performing computations on graphs, we also define V = the number of vertices in the graph being considered and E = the number of edges in the graph being considered. We will assume that a graph is represented as a sequence of V adjacency lists, where each list consists of a fixed amount of information about a vertex followed by a list of its incident edges. Given such a representation, we have $N = \Theta(V + E)$.

1.2. Previous results

Aggarwal and Vitter [1] first considered the problem of designing I/O-efficient algorithms. They gave several algorithms for basic problems such as sorting, permuting, and matrix operations. Their results were proved in a weaker computational model in which the external memory consists of a single large disk containing all N items, and the basic I/O step can transfer *any* D blocks of B items between main memory and external memory. Optimal sorting algorithms were proved for the more general D -disk model by Vitter and Shriver [7] (using a randomized algorithm) and by Nodine and Vitter [5] (using a deterministic algorithm). Nodine and Vitter [6] also extended their sorting results to apply to a variety of memory hierarchies. However, until recently, research in I/O-efficient algorithms centered on the fundamental problems of sorting, permuting, and the like.

Goodrich et al. [4] were the first to develop external-memory techniques that would apply to wider classes of algorithms — in their case, computational geometry. Later, Chiang et al. [3] opened up the area of external-memory graph algorithms. In most of this work, the data structures used by the algorithms were motivated by the particular problems that were being considered, rather than being the focus of the work. Arge [2] improved several of the results in these earlier papers by the introduction of the I/O-efficient *buffer tree*, which was the first I/O-efficient data structure to incorporate an amortized analysis for batched operations. Arge's buffer tree was developed to assist in the design of algorithms in computational geometry, supporting operations such as range searching. However, it also led to simple algorithms for sorting and some graph problems, generalizing some of the results of Chiang et al. [3].

1.3. Our results

In this paper, we continue the development of I/O-efficient data structures, and illustrate further how they can assist in the design of I/O-efficient algorithms. We will focus on algorithms for solving problems on large graphs, but just as the simple versions of the data structures we present have found widespread use, we expect that our I/O-efficient versions will also find applications in other areas.

The two data structures we present are I/O-efficient versions of a *heap* and a *tournament tree*. We show that an external-memory generalization of a binary heap can achieve amortized performance of $O(\frac{1}{DB} \log_{\frac{M}{B}} \frac{N}{B})$ I/O's per operation (insert, delete, or deletemin) over a sequence of operations on a heap containing at most N elements. This data structure has some similarities to, and the same performance bounds as, Arge's buffer tree [2]. However, we omit some of the geometric information that Arge included, and as a result we do not have to perform complicated tree-balancing operations. Our I/O-efficient heap immediately yields a heapsort with optimal I/O performance.

Our I/O-efficient tournament tree requires $O(\frac{1}{DB} \log_2 \frac{N}{B})$ I/O's per operation (delete, deletemin, or update) over a sequence of operations on a tournament tree of at most N elements. However, in addition to general deletions and deletemins, the tournament tree also supports an *update* (or *decrease-key*) operation that, given an element and a new key value, changes the element's key value if and only if the current key value is greater than the new one. This data structure aids in the design of I/O-efficient algorithms similarly to the way that Fibonacci heaps aid in the design of conventional algorithms.

These data structures, together with some new algorithmic techniques, lead to algorithms with improved I/O performance for several fundamental graph problems. Our results include:

- New algorithms for finding minimum spanning trees with nearly-optimal I/O performance — $O(\log B \cdot sort(E) + \log V \cdot scan(E))$ I/O's;
- New algorithms for breadth-first search and single-source shortest paths;
- A generalized deterministic algorithm for list ranking.

We point out here that, for brevity, all our proofs will be given for the single-disk external-memory model — that is, for the case when $D = 1$. All of our proofs generalize in a straightforward way to external memories consisting of multiple parallel disks; the details will be included in the full version of the paper.

The remainder of the paper is organized as follows. In Sections 2 and 3, we present our I/O-efficient data structures

and establish amortized bounds on their performance. Sections 4 and 5 present several improved algorithms for graph problems that use our I/O-efficient data structures. Finally, in Section 6 we discuss our conclusions and mention some open problems.

2. An I/O-efficient heap

2.1. Description of the data structure

We introduce a data structure called the *I/O-efficient heap*, which is the external-memory analogue of the usual heap or priority queue. It has the structure of a k -ary tree where k is $\sqrt{M/B}$. Each node holds a number of elements in sorted order, and any element held in a node is smaller than any other element held in any of its descendant nodes. To achieve I/O efficiency, we develop a scheme of lazy updates for this data structure. For this purpose, every node is allowed to hold between $\sqrt{MB}/2$ and $2\sqrt{MB}$ elements. When the number of elements in a node goes out of this range, we take remedial action. Similarly, we provide for a *buffer* at every node with capacity M . For a node v , elements that are to be sent down to the descendants of v are accumulated in its buffer. When the buffer becomes full, its contents are transferred to the children of v . These features allow us to avoid having to undertake expensive I/O to update a node and its children every time an element is to move up from or down to it. Such updates can now be batched together to significantly reduce the I/O cost per update.

An *I/O-efficient heap* is a tree with the following properties:

- Each node contains a *list* of elements in sorted order. This number of elements in the list is between $\sqrt{MB}/2$ and $2\sqrt{MB}$.
- Each node has $\sqrt{M/B}$ children.
- Each node has a *buffer* that can contain up to M elements.
- Elements have a field called the *key*. There is an order defined on elements on the basis of the comparison of their key fields, and any element contained in a node's list is smaller than any element contained in its buffer or in any of its descendant nodes.

The *cardinality* of a heap is the number of elements contained in it.

The standard heap operations *insert*, *delete*, and *deletemin* are supported, and are implemented using the following primitives:

emptylist takes care of the situation when a node's list contains over $2\sqrt{MB}$ elements. It takes all except the smallest $\sqrt{MB}$ elements and puts them in the node's buffer.

emptybuffer is used to empty a node's buffer by distributing its contents among its child nodes. If the buffer contains x elements, up to $x/\sqrt{MB}$ of the node's children are potential recipients of the buffer's contents. The distribution is done in such a way as to balance the number of elements stored in each of the node's children as well as possible.

fillup is executed whenever the number of elements in a node's list drops below $\sqrt{MB}/2$. First we empty the node's buffer, and then extract enough elements from the child nodes to raise the number of elements in the list to $\sqrt{MB}$. Treating the children's lists as a collection of lists we have to merge, we obtain a merged list of the required size and append it to the list to be refilled (efficient list merging schemes are known).

2.2. The I/O complexity of heap operations

Clearly, the height of a heap that never contains more than N elements will be $O(\log_{\frac{M}{B}} \frac{N}{B})$. Let l denote the height of such a heap, with the root at level l and the leaves at level 0.

Our objective is to obtain an amortized bound on the total number of I/O's in terms of the number of heap operations performed. The lower bound of $\Omega(\frac{N}{B} \log_{\frac{M}{B}} \frac{N}{B})$ on sorting [1] together with the bound on heap height implies that a cost of $O(l/B)$ per heap operation would be optimal.

The primitives *emptybuffer*, *emptylist*, and *fillup* perform all the I/O operations, each one moving a collection of elements one level down or up the heap. These functions are I/O efficient — that is, when they move x elements up or down one level, they need x/B I/O's to do so.

Using a straightforward charging scheme, the details of which we omit, we can show that the amortized cost of each insert, delete, and deletemin is $O(l/B)$ I/O's. Using the bound on l , the number of levels, we obtain the following theorem:

Theorem 1 *On an I/O-efficient heap with at most N elements, a sequence of k operations (insert, delete, and deletemin) requires at most $O(\frac{k}{B} \log_{\frac{M}{B}} \frac{N}{B})$ I/O's.* ■

These performance bounds for the I/O-efficient heap lead immediately to an optimal heap sorting algorithm for N elements, which consists of just N insertions followed by N deletemins:

Corollary 1 *On an I/O-efficient heap, Heapsort sorts N elements using $O(\text{sort}(N))$ I/O's.* ■

(Arge [2] achieved the same bound for sorting with a different data structure.)

3. An I/O-efficient tournament tree

The tournament tree is a common and useful data structure. It offers some advantages over a heap as there is a natural mapping of the numbers $1, \dots, N$ to the N items in a tournament tree, and this mapping is implicit in the way a tournament tree is organized. The additional information is often useful.

Imagine a knock-out tournament to be played among $N = 2^k$ players. It would involve k rounds and $2 \cdot N - 1$ matches. In round i , 2^{k-i} matches would be played, each resulting in the elimination of the player that loses. The tournament can be represented using a binary tree, where each leaf represents one player, and each internal node holds the winner of a match played between its two child nodes. The root contains the winner of the tournament. The resulting data structure is called the tournament tree. The completed chart of the Wimbledon draw (with the winner of each match filled in), for instance, is a tournament tree with 128 leaves. Tournament trees are useful in many comparison-based problems, where the elements are the ‘players’, and contests represent comparisons. For example, a tournament tree can be used to implement a priority queue.

An I/O-efficient version of the tournament tree is presented below. Later, we will demonstrate applications to the single-source shortest paths problem and breadth-first search that illustrate the usefulness of this data structure.

3.1. Description of the data structure

The I/O-efficient tournament tree houses N elements, each of which is a pair of the form (x, y) . The x values, ranging from 1 to N , identify the elements. The structure is organized according to comparisons on the y values of the elements. The y value of an element will also be known as its *key*; all elements initially have a key value of infinity.

The structure supports three operations: *update*, *delete* and *deletemin*. *deletemin* returns the element with the smallest key from the tree and deletes it from the tree. A *delete*(x) changes the key value of x to infinity, and removes (x, y) from internal nodes on the path from the root to the x ’s leaf, where y is the old key value of x . An *update* (or *decrease – key*) changes the key value of an element only if the new value is smaller than the old value.

The tree is balanced and consists of $\log_2 \frac{N}{B}$ levels. The structure of the tree is static: vertex numbers are mapped to leaves. Every node at any level except the lowest has two children. The x value of any element in the left child is smaller than that of any element in the right child. The M elements with the smallest keys among all the elements in the child nodes are duplicated in the parent node. Each node has an associated *buffer* of size M .

Initially, all elements have a key value of infinity. When an operation is performed, a *delete* or *update signal* is generated and inserted in the root. When a signal is inserted in a node, it may affect some elements held in that node. The values of those elements may have to be changed, and the signal may have to be propagated down to the appropriate child node. This is done in the following (lazy) way.

A *delete* signal always propagates to the lowest level. An *update* propagates down till it meets the old value in some node. If the old value is smaller than the new value, the signal is killed, else the value is changed and the signal propagates down. When a signal is inserted into the root (which is held in memory) the appropriate changes are made to elements held in the root and the signal added to the root buffer. When a buffer contains M signals we *empty* it. The emptying process involves distributing the signals among the two child nodes, applying to each child node its share of the signals (for which the child node has to be read in), and then adding to the child node’s buffer and signals that have to propagate down the tree. It is easy to see that

Observation 1 An empty operation requires $O(\frac{M}{B})$ I/O’s.

When the number of undeleted elements in the root is reduced to $\frac{M}{2}$, we *fillup* the root. The *fillup* of a node is done as follows: We extract from the child nodes as many elements as are required to restore the strength of the parent node to M elements. These elements are the ones with the smallest key values among all the elements contained in its child nodes (other than those already present in the parent node). Before this can be done, however, the child nodes must be updated with all the signals in the parent’s buffer. For this purpose, the parent buffer is emptied. If during the *fillup* any child node loses all its elements, that node is filled up before the parent can be filled up. At the end we check if any child node is depleted to a strength of less than $\frac{M}{2}$ elements: that node is in turn filled up. We observe that the cost of the whole operation is $O(\frac{M}{B})$ I/O’s.

3.2. The I/O complexity of tournament tree operations

We begin with the observation that, since the a tournament tree has $\frac{N}{M}$ leaf nodes, and every tree node has two children, the height of a tournament tree containing N elements is $O(\log_2 \frac{N}{M}) = O(\log_2 \frac{N}{B})$.

A straightforward charging scheme, involving the movement of elements and signals up and down the tree, yields the following amortized bound on the I/O requirements of the basic tournament tree operations (the details are omitted):

Theorem 2 On an I/O-efficient tournament tree with N elements, a sequence of k operations (*delete*, *deletemin*, and *update*) requires at most $O(\frac{k}{B} \log_2 \frac{N}{B})$ I/O’s. ■

The ability of the I/O-efficient tournament tree to perform updates to the key values of entries without knowing their current values will be used in Section 5 to help design new I/O-efficient algorithms for and breadth-first search and single-source shortest paths.

In the following sections we present improved I/O-efficient algorithms for several basic graph problems that use the data structures presented in Sections 2 and 3.

4. Improved minimum spanning tree algorithms

There are well-known $O(V \log V)$ -time algorithms for the minimum spanning tree problem. Those techniques, however, are not suitable for external-memory computation. An algorithm is presented here that uses $O(\log B \cdot \text{sort}(E) + \log V \cdot \text{scan}(E))$ I/O's to compute a minimum spanning tree.

First we present an algorithm for a restricted case in which there are no small-degree vertices in the graph. In the following, a vertex will be said to have a *large* degree if there are at least B edges incident to it. Otherwise, it is a *small*-degree vertex. The skeleton of the algorithm is familiar: it consists of a number of phases and in every phase, it uses the smallest-weight-edge on every vertex to get a set of connected components. Vertices in a component are then paired up for merging. The edges that are shrunk are designated MST edges, the adjacency lists of component vertices are merged to obtain the adjacency list of the new vertex and internal edges are removed from the combined adjacency list.

As we shall see, the number of pairs is a constant fraction of the number of vertices in that phase. Therefore, each phase reduces the number of vertices in the graph by a constant factor. This implies that the number of phases in the algorithm is $O(\log V)$. We shall also see that each phase requires no more than $O(\text{scan}(E))$ I/O's.

4.1. An algorithm for large-degree graphs

The graph $G = (V, E)$ is represented as a list of adjacency lists. At the head of each adjacency list is the smallest-weight edge incident on the vertex. Then follows the list of vertices of G which have merged to form this (hyper)vertex. This is followed by the rest of the edges, ordered by the vertex number of the destination.

In one phase, each adjacency list undergoes no more than one merge. A constant fraction of the lists undergo a merge. The process involves a scan of the list of lists to produce a new list of lists.

When the scan encounters a new adjacency list (for some vertex s not involved in a merge so far in this phase), we

try to merge it with another list. Let the smallest-weight-edge point to vertex u . u may have been merged into some vertex v . One lookup is needed to know v . If the adjacency list of v has not been involved in a merge in this phase, merge the two lists and put the result in the new list of lists being produced. The new vertex gets the smaller of the two vertex numbers. The smallest-weight-edge that led us to v is removed and included in a set of tree edges.

In case the vertex v has already merged with some vertex, we will try to merge s with another vertex that desires to merge with v , so that over a number of phases, v and all the vertices whose smallest-weight-edges point to v are merged together. If another such vertex t has already been encountered, it will be 'found waiting on v ', and can be discovered by a single lookup of v . If such a vertex t is indeed found, we merge t and s , remove t from the 'waiting', and delete the heavier of the two smallest-weight-edges and include it among the tree edges. If no vertex is waiting on v , we simply make s wait on v .

The graph thus produced by merging vertices may contain multiple edges between vertices. Merging two vertices with more than one edge between them gives rise to internal edges. While merging lists, we do some simple bookkeeping to make sure all internal edges are thrown away.

To analyze the algorithm's performance, we simply bound the number of I/O's used per phase and the number of phases required by the algorithm, and the desired bound on the algorithm follows.

Theorem 3 *The large-degree MST algorithm requires at most $O(\log V \cdot \text{scan}(E))$ I/O's.*

Proof: The algorithm involves no more than $\log V$ phases. This is because at the end of a phase, no more than one vertex can be waiting at any particular vertex, and if u is waiting on v , then v has been involved in a merge. It follows that each phase reduces the number of vertices by at least a third.

The number of I/O's involved in a phase is no more than $O(\text{scan}(E))$, since it is clear from the description of a phase that we require no more than two scans of any adjacency list, plus a constant number of block reads/writes per vertex. As every adjacency list is of length at least B , the total number of I/O's involved in these reads/writes is bounded from above by $O(\text{scan}(E))$. It follows that the total number of I/O's required is $O(\log V \cdot \text{scan}(E))$. ■

4.2. An algorithm for the general case

Now we present an algorithm that does not require all adjacency lists to be large.

We assume that small-degree vertices have smaller vertex numbers than large-degree vertices. Their adjacency lists will therefore occur before large adjacency lists in our

list of lists. This can be initially achieved by renumbering the vertices and reordering the list of adjacency lists. This requires $O(1)$ sorts on the set of edges. Our algorithm then maintains this property throughout.

First of all, we do a scan of the list of adjacency lists to get the smallest-weight edge for every vertex. Using these edges, we use the following scheme to pair up the vertices for merging:

First consider all the edges going from higher-numbered vertices to lower-numbered vertices. For every edge (i, j) , create a pair (i, j) and insert the pairs in a heap, treating j as the primary and i as the secondary key (as in radix sort). We also insert a pair $(0, u)$ for every vertex u . Now, using *deletemin* successively, we obtain the vertices pointing to the smallest numbered vertex u (and also the vertex u if the entry $(0, u)$ has not already been deleted). We construct pairs out of these vertices. If an odd number of vertices are involved, the pairing process leaves one unpaired vertex, which is left *waiting* on u . Whenever a child vertex v is paired up, we delete the entry $(0, v)$ from the heap. This ensures that v is not paired up with one of its children.

The pairing step is then repeated using edges directed from lower-numbered vertices to higher-numbered vertices. But we make sure that the entry $(0, v)$ is not inserted in the heap for any vertex v which has already been paired up in the previous step.

Each of these two pairing steps may leave one vertex waiting on a vertex u . If in the end there are two vertices waiting on u , they are paired together. If there is one vertex waiting on u , it is added to the pair that contains u , to form a triple.

Now that we have determined which vertices are to be merged together, all that remains is the actual task of merging. The pairs (or triples) that contain at least one large-degree vertex are merged as in the restricted-case algorithm above, and the resulting vertex is named after the smallest-numbered large-degree constituent vertex. To merge pairs (or triples) containing small vertices only, we give every adjacency list a *destination*, that is the vertex number of the vertex to be formed as a result of the merge. Then all the small adjacency lists are sorted together by destination. This ensures that the adjacency lists that merge together are placed one after the other. Then the merging can be accomplished in one scan. Finally, the small adjacency lists are sorted by size, and the small-degree vertices renumbered to ensure that any vertices whose adjacency lists become large as a result of the last merge are placed after the small-degree vertices. This completes one phase of our algorithm. Once again, $O(\log V)$ phases are required by the algorithm, since a phase reduces the number of vertices by a fraction.

We now analyze the performance of the algorithm. As noted before, the initial reordering takes $O(\text{sort}(E))$ I/O's.

Let us consider a phase in which the residual graph G'

has V' vertices and E' edges. Consider the process of extracting the smallest-weight edges from all the vertices and using them to pair vertices up. Extracting the edges requires just a scan of the list of adjacency lists. Then, we need $O(V')$ heap operations, and (let us say) a constant number of scans and sorts on $O(V')$ items to order and combine the results of the two pairing steps and so forth. The I/O cost of all this is $O(\frac{V'}{B} \log_{\frac{M}{B}} \frac{V'}{B})$. As V' goes down by at least a constant fraction in every phase, these terms add up to just $O(\frac{V}{B} \log_{\frac{M}{B}} \frac{V}{B}) = O(\text{sort}(V))$ over the whole algorithm.

The merging process requires $O(\text{scan}(E'))$ I/O's as before, except for the extra work done on the smaller adjacency lists. Let E'_s be the total number of edges in all small adjacency lists in this phase. The extra work done in dealing with the small adjacency lists is $O(\text{sort}(E'_s))$, as it involves a constant number of sorts on the small lists. Let each edge thus sorted be charged $O(\frac{1}{B} \log_{\frac{M}{B}} \frac{V}{B})$.

What does this cost add up to over the entire algorithm? Let us for the moment ignore adjacency lists with one or two edges at this point: they can separately be taken care of - we omit the details. We can now say that an edge is charged only when it is involved in a merge that makes it the member of a bigger adjacency list: one that is bigger by at least a constant factor. Thus, an edge can only be charged $O(\log B)$ times in this fashion before it becomes a part of an adjacency list of size B or more, after which it is never involved in any sorting. The total cost charged to all the edges in this fashion is therefore no more than $O(\log B \cdot \frac{E}{B} \log_{\frac{M}{B}} \frac{E}{B}) = O(\log B \cdot \text{sort}(E))$ over the entire algorithm. So we have

Theorem 4 *The general-case MST algorithm requires at most $O(\log B \cdot \text{sort}(E) + \log V \cdot \text{scan}(E))$ I/O's.* ■

This I/O performance is better than that of the best previously-known algorithm, due to Chiang et al. [3], except when the graph is extremely dense ($E = \Theta(V^2)$).

5. Improved algorithms for other graph problems

In this section we present improved external memory algorithms for list ranking, single-source shortest paths, and breadth-first search.

5.1. A generalized deterministic list-ranking algorithm

Chiang et al. [3] suggested a method to deterministically rank an N -vertex linked list using $O(\text{sort}(N))$ I/O's, by splicing out entire chains of forward- or backward-pointing edges, but it requires that M/B be sufficiently large for the algorithm to work. We present an alternate algorithm using an I/O-efficient heap that removes this restriction on B .

Another way to remove this restriction is suggested by Arge [2].

First, we show that given a set of lists constructed out of N vertices in which all the edges go from lower numbered vertices to higher numbered vertices (*forward edges*), we can in $O(\text{sort}(N))$ I/O's rank all the lists. Insert each edge into a heap, with its source as the key. Extract the first edge (u, v) from the heap; it must be the head of a list, so give it rank 1, and insert a special element $(v, -2)$ into the heap. Thereafter, each time we remove an edge (v, w) from the heap, if it is not preceded by an edge of the form $(v, -r)$, we give it rank 1 and insert $(w, -2)$ into the heap; if it is preceded by an edge $(v, -r)$, we give it rank r and insert $(w, -(r+1))$ into the heap. (The head of the list could also be encoded into the element and passed along, if desired, for splicing purposes.) Clearly, this ranking procedure will take $O(\text{sort}(N))$ steps, since there are $O(N)$ heap operations involved.

It follows that for any linked list, we can rank independently each sublist of forward edges and each sublist of backward edges, and also splice out all of their internal vertices, in $O(\text{sort}(N))$ I/O's. The remaining list will consist of alternating forward and backward edges. Once the list is known to consist of alternating forward and backward edges, it is easy to splice out a constant fraction of the vertices using $O(\text{sort}(N))$ I/O's. The resulting sublist is then ranked recursively, and the final ranking is computed using $O(\text{sort}(N))$ additional I/O's. This yields the desired result.

Theorem 5 *List ranking can be done deterministically using $O(\text{sort}(N))$ I/O's.* ■

(Arge [2] reports the same bound, using a different data structure.)

5.2. An algorithm for single-source shortest paths

Using our tournament tree, we can obtain an I/O-efficient version of Dijkstra's algorithm. The tournament tree replaces the priority queue in the original algorithm. Initially we let our V vertices be initialized to infinite distance except for the source which is at distance zero. Using the distance as the key, we construct a tournament tree of these V elements. For the next V steps, we do a *deletemin* on the tree, read the adjacency list of the vertex obtained, and for every edge $e = (u, v)$ in the adjacency list (except when v is the vertex that precedes u on the shortest path from the source to u : the identity of that vertex v can be stored in the tournament tree entry for u) we update the key of vertex v to $x + w(e)$, where x is the distance of vertex u from the source and $w(e)$ the weight of edge e . It is easy to see that this takes $O(V + \frac{E}{B} \log_2 \frac{E}{B})$ I/O's: $O(V + \frac{E}{B})$ to read all

the adjacency lists, and $O(\frac{E}{B} \log_2 \frac{E}{B})$ for all the tournament tree operations.

An auxiliary heap is used to correct for spurious updates of nodes that have already been deleted since their shortest paths have been found (we omit the details in this abstract). The $O(E)$ added heap operations require an additional $O(\frac{E}{B} \log_2 \frac{E}{B})$ I/O's. Our result follows immediately.

Theorem 6 *Single-source shortest paths can be found using $O(V + \frac{E}{B} \log_2 \frac{E}{B})$ I/O's.* ■

We believe that this is the best known upper bound on I/O's for this problem.

5.3. An algorithm for breadth-first search

Our breadth-first search algorithm is similar to the algorithm for single-source shortest paths just discussed.

We start with a tournament tree containing the V vertices and their key values: the root initialized to zero, the rest to infinity. We keep a counter initialized to 0. The basic step of the algorithm is as follows: the vertex v with the smallest key is extracted from the tournament tree. Let $v_1, v_2, \dots, v_l$ be vertices in the adjacency list of v , excluding the parent of v in the BFS tree. Their keys are updated to $k, k+1, k+2, \dots, k+l$ respectively, where k is the value of the counter. The counter is changed to $k+l$.

The scheme will visit every vertex once, and visit vertices in BFS order. However, there is a problem similar to the one we encountered in our algorithm to find single-source shortest paths. Again, an auxiliary heap corrects for spurious updates of vertices that have already been visited and deleted (we omit the details).

The resulting algorithm performs a total of $O(E)$ tree and heap operations, in addition to reading each adjacency list once. This yields the following bound:

Theorem 7 *Breadth-first search can be performed using $O(V + \frac{E}{B} \log_2 \frac{E}{B})$ I/O's.* ■

As for single-source shortest paths, we believe that this is the best known upper bound on I/O's for this problem.

6. Conclusions and open questions

We have presented two I/O-efficient data structures — the I/O-efficient heap and tournament tree — and demonstrated how their use can simplify the design of I/O-efficient graph algorithms. Both of these data structures support all their basic operations: the heap with an amortized cost of $O(\frac{1}{DB} \log_2 \frac{N}{B})$ I/O's per operation, and the tournament tree with an amortized cost of $O(\frac{1}{DB} \log_2 \frac{N}{B})$ I/O's per operation on a data structure of at most N items.

The I/O-efficient heap led immediately to simple descriptions of optimal algorithms for sorting and list-ranking, while the update operation available in the I/O-efficient tournament tree proved useful in the design of algorithms for breadth-first search and single-source shortest paths. We also obtained an algorithm for finding minimum spanning trees with improved I/O efficiency on all but the densest graphs. As is the goal of data structures, the I/O-efficient heap and tournament tree both simplified the descriptions of external-memory graph algorithms and led to improved efficiency.

There are still many open questions, including:

- Can minimum spanning trees be found in as few as $O(\text{sort}(E))$ I/O's? How about in $O(\text{sort}(V))$, which was shown to be a lower bound by Chiang et al. [3]?
- Can the additive V terms be removed from the running times of the algorithms for breadth-first search and single-source shortest paths?
- Can depth-first search be performed as efficiently as breadth-first search? (We expect that this is the case.)
- Can a tournament tree be designed with an amortized I/O complexity of $O(\frac{1}{B} \log_{\frac{M}{B}} \frac{N}{B})$ I/Os per operation, instead of $O(\frac{1}{B} \log_2 \frac{N}{B})$ I/Os per operation?
- In what other areas can these data structures lead to improved algorithms? What other data structures would it be helpful to transfer to an external-memory setting?

References

- [1] A. Aggarwal and J. S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [2] L. Arge. The buffer tree, a new technique for optimal I/O-algorithms. In *Proceedings of the Fourth Workshop on Algorithms and Data Structures*, pages 334–345, 1995.
- [3] Y.-J. Chiang, M. Goodrich, E. Grove, R. Tamassia, D. Vengroff, and J. Vitter. External memory graph algorithms. In *Proceedings of the Sixth ACM-SIAM Symposium on Discrete Algorithms*, pages 139–149, 1995.
- [4] M. Goodrich, J.-J. Tsay, D. Vengroff, and J. Vitter. External-memory computational geometry. In *Proceedings of the 34th Annual IEEE Symposium on Foundations of Computer Science*, pages 714–723, 1993.
- [5] M. Nodine and J. Vitter. Large-scale sorting in parallel memories. In *Proceedings of the Third Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 29–39, 1991.
- [6] M. Nodine and J. Vitter. Deterministic distribution sort in shared and distributed memory multiprocessors. In *Proceedings of the Fifth ACM Symposium on Parallel Algorithms and Architectures*, pages 120–129, 1993.
- [7] J. Vitter and E. Shriver. Algorithms for parallel memory I: two level memories. *Algorithmica*, 12(2):110–147, 1994.